LAPORAN PROJECT BASED LEARNING

“Implementasi AWS Cloud untuk Analisis dan Prediksi Berbasis Historis Kejadian Banjir di DKI Jakarta”

Disusun Untuk Memenuhi Evaluasi Tengah Semester Mata Kuliah Big Data

DOSEN PENGAMPU:

Masti Fatchiyah Maharani, S.Kom.,

M.Cs Dr. Eng. Ir. Dwi Arman Prasetya, ST., MT., IPU., Asean. Eng

DISUSUN OLEH:

1. **Maria Dwi Kurniasih** **(24083010003)**
2. **Hizkia Samhan Rezayoshi** **(24083010019)**
3. **Esthi Nurani Sri Handayani** **(24083010081)**

PROGRAM STUDI SAINS DATA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN “VETERAN” JATIM

2025

DAFTAR ISI

DAFTAR ISI 1
BAB I 2
PENDAHULUAN 2
1.1. Latar Belakang 2
1.2. Rumusan Masalah 3
1.3. Tujuan 4
BAB II 5
KAJIAN TEORI 5
2.1 Definisi Cloud Computing 5
2.2 Model Layanan Cloud (IaaS, PaaS, SaaS) 5
2.3 Karakteristik Cloud Computing 6
2.4 Keuntungan Menggunakan AWS untuk Studi Kasus 6
BAB III 8
HASIL DAN PEMBAHASAN 8
3.1. Arsitektur Diagram 8
3.2. Data Source dan Collecting 8
3.2.1 Sumber dataset 8
3.3. Data Storage dengan AWS S3 14
3.4. Data Processing dengan SageMaker 17
3.5. DATA ACCESSING & DASHBOARD 24
BAB IV 28
KESIMPULAN 28
4.1 Insight yang Diperoleh dari Analisis Data 28
4.2 Efektivitas Penggunaan AWS untuk Studi Kasus 28
4.3 Kendala yang Dihadapi dan Solusinya 28
4.4 Pembelajaran tentang Cloud Computing 29

BAB I
PENDAHULUAN

1.1. Latar Belakang

DKI Jakarta, merupakan pusat gravitasi ekonomi dan pemerintahan Indonesia, secara konsisten menghadapi ancaman eksistensial berupa bencana banjir yang semakin kompleks setiap tahunnya. Secara geografis, posisi Jakarta yang berada di dataran rendah dengan 40% wilayahnya berada di bawah permukaan laut, menjadikannya muara bagi 13 sungai besar dari wilayah hulu. Fenomena ini menciptakan tantangan hidrologis yang unik, di mana kota harus menghadapi tiga ancaman sekaligus banjir kiriman akibat debit air tinggi dari wilayah Bogor dan Puncak, banjir lokal yang dipicu oleh curah hujan ekstrem di tengah keterbatasan sistem drainase perkotaan, serta banjir rob akibat kenaikan permukaan laut dan penurunan muka tanah (*land subsidence*) yang progresif di wilayah utara. Selama ini, manajemen bencana cenderung bersifat reaktif dan terfragmentasi, di mana pengambilan keputusan seringkali hanya didasarkan pada pemantauan visual atau pelaporan manual yang memiliki jeda waktu signifikan, sehingga peluang untuk melakukan mitigasi dini sering kali terlewatkan.

Pengolahan data konvensional menggunakan infrastruktur *on-premise* seringkali menghadapi kebutuhan akan sistem pemantauan yang cerdas menjadi semakin mendesak seiring dengan meningkatnya volume dan variabilitas data lingkungan dalam skalabilitas dan biaya operasional yang tinggi, terutama saat harus menangani pemutaran data secara tiba-tiba ketika cuaca ekstrem terjadi. Penerapan teknologi komputasi awan melalui platform Amazon Web Services (AWS) menghadirkan pendekatan baru dalam manajemen data bencana yang lebih efektif dan dapat diukur. Dengan memanfaatkan ekosistem AWS, inisiatif ini dirancang untuk membangun saluran data yang saling terhubung, dimulai dari pengumpulan dataset sejarah kejadian banjir di DKI Jakarta yang kemudian akan dibersihkan menggunakan pustaka Pandas. Sistem ini menggunakan layanan Amazon S3 sebagai penyimpanan utama yang dibagi ke dalam folder /raw untuk data kasar dan folder /processed untuk data yang sudah ditransformasi.

Pendekatan ini menjamin pengelolaan aset data yang terstruktur, aman, dan memungkinkan akses yang mudah melalui pengaturan penuh AWS Identity and Access Management (IAM).

Fokus utama dari proyek ini adalah mentransformasi data historis tersebut menjadi wawasan prediktif dengan memanfaatkan kekuatan komputasi pada Amazon EC2 dan kapabilitas analisis dari Amazon SageMaker. Dengan menggunakan algoritma *machine learning*, sistem dikembangkan untuk memahami pola banjir di masa lalu berdasarkan variabel seperti lokasi dan tinggi air untuk menghasilkan prediksi yang lebih presisi. Keunggulan elastisitas AWS memastikan bahwa proses pengolahan data masif dan pelatihan model tetap berjalan dengan performa optimal meskipun beban kerja mencapai puncaknya. Dengan tersedianya informasi yang diolah secara cepat dan divisualisasikan melalui *library* Matplotlib atau Seaborn, para pembuat keputusan dapat memperoleh gambaran yang jelas mengenai distribusi risiko banjir. Laporan proyek ini akan membahas secara mendalam arsitektur sistem, integrasi layanan S3 dan EC2, hingga efektivitas model prediksi yang dihasilkan, guna memberikan kontribusi nyata dalam upaya mitigasi bencana dan perlindungan keselamatan penduduk DKI Jakarta di masa depan.

1.2. Rumusan Masalah

Berdasarkan latar belakang sebelumnya, maka rumusan masalah dalam proyek ini adalah:

1. Bagaimana merancang dan mengimplementasikan arsitektur *data pipeline* yang terintegrasi menggunakan layanan Amazon S3 sebagai media penyimpanan dan Amazon EC2/SageMaker sebagai unit pemrosesan data banjir?
2. Bagaimana efektivitas penggunaan library boto3 dalam menghubungkan proses analisis data di *cloud* dengan penyimpanan data mentah (*raw*) dan data terolah (*processed*) di AWS S3?
3. Sejauh mana model algoritma yang dikembangkan (seperti *Machine Learning* atau analisis historis) mampu mengidentifikasi pola kejadian banjir di DKI Jakarta untuk memberikan wawasan prediktif yang presisi?

4. Bagaimana tingkat efisiensi penggunaan infrastruktur AWS Cloud dalam menangani skalabilitas data dan pemrosesan informasi secara cepat dibandingkan dengan metode konvensional?

1.3. Tujuan

Adapun tujuan yang ingin dicapai melalui pelaksanaan proyek ini adalah:

1. Membangun sistem pipeline data berbasis cloud-native yang mencakup tahap pengumpulan, pembersihan, penyimpanan, hingga analisis data secara terotomatisasi di ekosistem AWS.
2. Mengoptimalkan penggunaan layanan Amazon S3 untuk penyimpanan dataset banjir yang aman, terstruktur (menggunakan folder /raw dan /processed), serta memiliki fitur versioning.
3. Menghasilkan visualisasi data yang informatif untuk mempermudah interpretasi distribusi data dan hasil analisis utama mengenai kejadian banjir di DKI Jakarta.
4. Membuktikan keunggulan elastisitas dan skalabilitas AWS dalam memproses data dalam jumlah besar guna mendukung pengambilan keputusan mitigasi bencana yang lebih proaktif dan cepat.
5. Memberikan kontribusi teknis berupa kerangka kerja (framework) analisis data banjir yang dapat digunakan sebagai referensi untuk meningkatkan ketahanan kota (urban resilience) berbasis teknologi digital.

BAB II
KAJIAN TEORI

2.1 Definisi Cloud Computing

Cloud computing merupakan model komputasi yang memungkinkan pengguna untuk mengakses sumber daya komputasi seperti server, penyimpanan, database, jaringan, perangkat lunak, dan layanan lainnya melalui internet secara fleksibel dan sesuai kebutuhan. Menurut National Institute of Standards and Technology (NIST), cloud computing adalah model yang memungkinkan akses jaringan yang mudah, sesuai permintaan (on-demand), ke kumpulan sumber daya komputasi bersama yang dapat dengan cepat disediakan dan dirilis dengan usaha manajemen minimal atau interaksi dengan penyedia layanan .

Dengan adanya cloud computing, pengguna tidak perlu lagi menyediakan infrastruktur fisik secara langsung karena seluruh layanan telah disediakan oleh penyedia cloud seperti Amazon Web Services (AWS).

2.2 Model Layanan Cloud (IaaS, PaaS, SaaS)

Cloud computing memiliki tiga model layanan utama, yaitu:

1. **Infrastructure as a Service (IaaS)**

   IaaS menyediakan infrastruktur dasar seperti server virtual, penyimpanan, dan jaringan. Pengguna memiliki kontrol penuh terhadap sistem operasi dan aplikasi yang dijalankan. Contoh layanan IaaS adalah Amazon EC2 yang memungkinkan pengguna untuk menjalankan server virtual sesuai kebutuhan

2. **Platform as a Service (PaaS)**

   PaaS menyediakan platform untuk mengembangkan, menjalankan, dan mengelola aplikasi tanpa perlu mengelola infrastruktur secara langsung. Pengguna hanya fokus pada pengembangan aplikasi. Contoh layanan ini adalah AWS SageMaker yang digunakan untuk pengolahan data dan machine learning .

3. **Software as a Service (SaaS)**

   SaaS merupakan layanan perangkat lunak yang dapat diakses langsung melalui nternet tanpa instalasi lokal. Pengguna hanya perlu menggunakan

aplikasi yang telah disediakan. Contohnya adalah Google Docs atau Microsoft Office 365.

2.3 Karakteristik Cloud Computing

Menurut NIST, cloud computing memiliki lima karakteristik utama, yaitu:

1. **On-Demand Self-Service**

   Pengguna dapat mengakses dan mengelola sumber daya komputasi secara otomatis tanpa interaksi langsung dengan penyedia layanan .

2. **Broad Network Access**

   Layanan cloud dapat diakses melalui jaringan menggunakan berbagai perangkat seperti laptop, smartphone, dan tablet.

3. **Resource Pooling**

   Sumber daya komputasi dikumpulkan dan dibagi untuk melayani banyak pengguna dengan sistem multi-tenant .

4. **Rapid Elasticity**

   Sumber daya dapat ditambah atau dikurangi dengan cepat sesuai kebutuhan pengguna .

5. **Measured Service**

   Penggunaan sumber daya dapat diukur dan pengguna hanya membayar sesuai dengan penggunaan (pay-as-you-go).

2.4 Keuntungan Menggunakan AWS untuk Studi Kasus

Amazon Web Services (AWS) merupakan salah satu penyedia layanan cloud computing yang banyak digunakan karena fleksibilitas dan skalabilitasnya. Dalam studi kasus analisis kejadian banjir di DKI Jakarta, AWS memberikan beberapa keuntungan, antara lain:

1. **Skalabilitas Tinggi**

   AWS memungkinkan pengguna untuk menyesuaikan kapasitas sumber daya sesuai kebutuhan analisis data yang dilakukan.

2. **Penyimpanan Data yang Aman dan Terstruktur**

   Layanan Amazon S3 memungkinkan penyimpanan data dalam jumlah besar dengan tingkat keamanan tinggi serta mendukung versioning.

3. **Kemudahan Integrasi Layanan**

   AWS menyediakan berbagai layanan yang dapat terintegrasi, seperti EC2 untuk pemrosesan data dan S3 untuk penyimpanan, sehingga memudahkan pembangunan pipeline data end-to-end .

4. **Efisiensi Biaya**

   Model pembayaran berbasis penggunaan (pay-as-you-go) memungkinkan pengguna hanya membayar sumber daya yang digunakan.

5. **Dukungan Analisis Data**

   AWS menyediakan layanan yang mendukung analisis data, seperti SageMaker dan integrasi dengan Python, sehingga mempermudah proses eksplorasi dan visualisasi data.

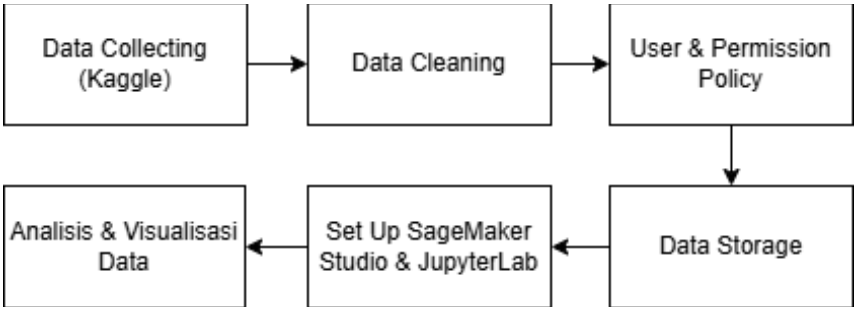
BAB III
HASIL DAN PEMBAHASAN

3.1. Arsitektur Diagram

Arsitektur sistem pada penelitian ini dirancang menggunakan pendekatan pipeline data berbasis cloud computing dengan memanfaatkan layanan Amazon Web Services (AWS). Proses dimulai dari tahap pengumpulan data banjir yang kemudian dilakukan pembersihan data menggunakan Python dan library Pandas.

Data yang telah dibersihkan selanjutnya disimpan ke dalam Amazon S3 pada folder /raw sebagai data mentah. Selanjutnya, data diakses oleh Amazon EC2 menggunakan library boto3 untuk dilakukan proses analisis dan transformasi data. Hasil pengolahan kemudian disimpan kembali ke Amazon S3 pada folder /processed.

Tahap akhir adalah visualisasi data menggunakan library Matplotlib atau Seaborn untuk menampilkan hasil analisis dalam bentuk grafik. Seluruh akses terhadap layanan AWS dikontrol menggunakan AWS Identity and Access Management (IAM) untuk menjaga keamanan dan pengaturan hak akses pengguna.



3.2. Data Source dan Collecting

3.2.1 Sumber dataset

Data yang digunakan dalam penelitian ini bersumber dari dataset historis kejadian banjir di wilayah DKI Jakarta pada tahun 2023 yang diperoleh melalui platform publik pada website Satu Data Jakarta. Pemilihan sumber data ini didasarkan pada ketersediaan data terbuka yang komprehensif mengenai titik-titik genangan, sehingga memungkinkan proses analisis pola historis dan pelatihan model prediksi dilakukan secara valid.

Penggunaan data sekunder dari API publik atau penyedia dataset terbuka ini menjamin transparansi dan aksesibilitas data bagi pengembangan *pipeline* berbasis *cloud*.

Dataset ini tersimpan dalam format file .csv (*Comma Separated Values*) dengan menggunakan karakter titik koma (;) sebagai pemisah antar kolom (*delimiter*). Penggunaan format ini dipilih karena sifatnya yang ringan, bersifat *open-standard*, dan memiliki kompatibilitas yang sangat tinggi dengan berbagai layanan AWS seperti Amazon S3 untuk penyimpanan serta pustaka pemrosesan data seperti Pandas di Amazon EC2 atau SageMaker.

Dataset ini mencakup rekaman kejadian banjir di wilayah DKI Jakarta sepanjang tahun 2023 yang meliputi total 194 baris data (observasi) dan 15 kolom. Jumlah data ini memberikan sampel yang cukup representatif untuk melakukan analisis pola frekuensi banjir, identifikasi wilayah yang paling sering terdampak, serta menjadi basis data historis bagi model *machine learning* dalam memprediksi kejadian serupa di masa mendatang. Dataset ini memiliki 15 kolom (fitur) utama yang memberikan informasi komprehensif dari aspek lokasi, waktu, hingga dampak fisik banjir. Berikut adalah rincian fitur tersebut:

No	Nama Kolom	Tipe Data	Deskripsi Singkat
1	Periode_data	int64	Tahun periode pengambilan data
2	bulan	int64	Bulan terjadinya kejadian banjir
3	wilayah	object	Nama wilayah kota administrasi di DKI Jakarta.
4	kecamatan	object	Nama wilayah tingkat kecamatan.
5	kelurahan	object	Nama wilayah tingkat kelurahan.

6	jumlah_rw_terdampak	int64	Akumulasi jumlah RW yang tergenang air di wilayah tersebut.
7	jumlah_kk_terdampak	int64	Jumlah Kepala Keluarga yang terdampak langsung oleh banjir.
8	jumlah_jiwa_terdampak	int64	Total jumlah individu/jiwa yang terdampak bencana.
9	jumlah_kejadian	int64	Frekuensi atau jumlah insiden banjir yang tercatat.
10	jumlah_korban_meninggal	int64	Jumlah korban jiwa akibat bencana banjir.
11	jumlah_korban_luka	int64	Jumlah warga yang mengalami cedera atau luka-luka.
12	jumlah_pengungsi	int64	Total warga yang harus dievakuasi ke tempat aman.
13	jumlah_tempat_pengungsian	int64	Jumlah titik posko pengungsian yang diaktifkan.
14	nilai_kerugian	int64	Estimasi kerugian materiil (dalam mata uang) akibat banjir.
15	rata_rata_ketinggian_air_num	float64	Angka rata-rata ketinggian genangan air dalam satuan cm.

3.2.2 Data Preprocessing

Preprocessing data dilakukan untuk menjaga kualitas data supaya analisis yang dilakukan memiliki hasil yang baik dan akurat. Preprocessing diawali dengan melakukan load data.

```python
import pandas as pd
import re
import numpy as np

df = pd.read_csv('banjir.csv', delimiter=';')

# ==========================================
# TAHAP SEBELUM PREPROCESSING
# ==========================================
print("=== PREVIEW 5 BARIS PERTAMA (SEBELUM PREPROCESSING) ===")
print(df.head().to_string())
```

```
=== PREVIEW 5 BARIS PERTAMA (SEBELUM PREPROCESSING) ===
   periode_data  bulan          wilayah         kecamatan               kelurahan jumlah_rata_rata_ketinggian_air  jumlah_rw_terdampak  jumlah_kk_terdampak
0        202303      2  JAKARTA SELATAN    KEBAYORAN LAMA          GROGOL SELATAN                          50 CM                    2                    0
1        202303      2  JAKARTA SELATAN    KEBAYORAN LAMA  KEBAYORAN LAMA SELATAN                          20 CM                    1                    0
2        202303      2  JAKARTA SELATAN  MAMPANG PRAPATAN            PELA MAMPANG                   25 S.D 40 CM                    1                    0
3        202303      2  JAKARTA SELATAN  MAMPANG PRAPATAN            TEGAL PARANG                          40 CM                    1                    0
4        202303      2  JAKARTA SELATAN  MAMPANG PRAPATAN          KUNINGAN BARAT                   30 S.D 60 CM                    3                    0
```

Langkah pertama yang dilakukan adalah Data Ingestion atau Load Data, yaitu memuat dataset dari file banjir.csv ke dalam struktur DataFrame Pandas untuk melihat gambaran awal isi data, seperti lokasi kejadian dan statistik dampak banjir. Terlihat bahwa dataset ini memiliki kolom ketinggian air yang masih berupa teks (seperti "25 S.D 40 CM"), sehingga memerlukan pembersihan lebih lanjut agar bisa dianalisis secara numerik.

Selanjutnya, proses beralih ke tahap Data Cleaning yang berfokus pada pengecekan *missing values*. Di sini, program menghitung jumlah sel yang kosong di setiap kolom serta menghitung total persentasenya terhadap keseluruhan data untuk memastikan kualitas informasi yang tersedia. Di bagian akhir, terlihat persiapan sebuah fungsi bernama hitung_rata_rata_air. Fungsi ini dirancang untuk melakukan Data Transformation, yaitu mengekstrak angka dari format teks yang tidak seragam menggunakan pustaka re (Regular Expression) agar nilai ketinggian air dapat dikonversi menjadi angka rata-rata agar siap digunakan ke proses selanjutnya.

```python
print("=== PENGECEKAN MISSING VALUES ===")
missing = df.isnull().sum()
print(missing)
print("-" * 40)
print(f"Total Missing Values: {total_missing} ({ (total_missing/total_cells)*100 }%)")
print("-" * 40)

def hitung_rata_rata_air(text):
    text = str(text).upper()
    if 'TIDAK ADA BANJIR' in text or text == '0':
        return 0.0

    angka = re.findall(r'\d+', text)
    if angka:
        values = [int(x) for x in angka]
        return sum(values) / len(values)
    return 0.0

cols_kategori = ['wilayah', 'kecamatan', 'kelurahan']
df[cols_kategori] = df[cols_kategori].fillna('TIDAK DIKETAHUI')

cols_numerik = [
    'jumlah_rw_terdampak', 'jumlah_kk_terdampak', 'jumlah_jiwa_terdampak',
    'jumlah_kejadian', 'jumlah_korban_meninggal', 'jumlah_korban_luka',
    'jumlah_pengungsi', 'jumlah_tempat_pengungsian', 'nilai_kerugian'
]
df[cols_numerik] = df[cols_numerik].fillna(0)
df['rata_rata_ketinggian_air_num'] = df['jumlah_rata_rata_ketinggian_air'].apply(hitung_rata_rata_air)
df[cols_numerik] = df[cols_numerik].astype(int)
df_final = df.drop(columns=['jumlah_rata_rata_ketinggian_air'])

# Menampilkan 10 baris pertama untuk kolom hasil perhitungan
print("--- Sampel Hasil Perhitungan Ketinggian Air ---")
print(df_final[['wilayah', 'kecamatan', 'rata_rata_ketinggian_air_num']].head(10))
```

Gambar tersebut menunjukkan tahap lanjutan dari Data Preprocessing, di mana fokus utamanya adalah penanganan data hilang (*missing values*) dan transformasi fitur numerik. Program mengidentifikasi nilai yang kosong dan melakukan pengisian (*imputation*) secara sistematis untuk kolom kategori seperti wilayah, kecamatan, dan kelurahan, nilai yang hilang diisi dengan label "TIDAK DIKETAHUI", sementara untuk kolom numerik seperti jumlah warga terdampak hingga nilai kerugian, nilai yang kosong diisi dengan angka "0". Langkah ini memastikan bahwa dataset tetap utuh dan tidak menyebabkan error saat dilakukan perhitungan statistik pada tahap berikutnya.

Selain pengisian data kosong, program tersebut menerapkan fungsi hitung_rata_rata_air untuk mengatasi kompleksitas format pada kolom ketinggian air. Fungsi ini menggunakan teknik Regular Expression (regex) untuk mencari angka di dalam string (seperti "25 S.D 40 CM") dan menghitung nilai rata-ratanya secara otomatis, jika teks berisi keterangan "TIDAK ADA BANJIR", maka fungsi akan mengembalikan nilai 0.0. Hasil dari proses ini disimpan ke dalam kolom baru bernama rata_rata_ketinggian_air_num, sementara kolom asli yang berbentuk teks dihapus menggunakan perintah df.drop. Proses diakhiri dengan

menampilkan sampel sepuluh baris pertama untuk memvalidasi bahwa konversi data dari teks ke numerik telah berhasil dilakukan dengan benar.

```
=== PENGECEKAN MISSING VALUES ===
periode_data                        0
bulan                               0
wilayah                             0
kecamatan                           0
kelurahan                           0
jumlah_rata_rata_ketinggian_air     0
jumlah_rw_terdampak                 0
jumlah_kk_terdampak                 0
jumlah_jiwa_terdampak               0
jumlah_kejadian                     0
jumlah_korban_meninggal             0
jumlah_korban_luka                  0
jumlah_pengungsi                    0
jumlah_tempat_pengungsian           0
nilai_kerugian                      0
dtype: int64
----------------------------------------
Total Missing Values: 0 (0.0%)
----------------------------------------
--- Sampel Hasil Perhitungan Ketinggian Air ---
           wilayah         kecamatan  rata_rata_ketinggian_air_num
0  JAKARTA SELATAN    KEBAYORAN LAMA                          50.0
1  JAKARTA SELATAN    KEBAYORAN LAMA                          20.0
2  JAKARTA SELATAN  MAMPANG PRAPATAN                          32.5
3  JAKARTA SELATAN  MAMPANG PRAPATAN                          40.0
4  JAKARTA SELATAN  MAMPANG PRAPATAN                          45.0
5  JAKARTA SELATAN      PESANGGRAHAN                          30.0
6  JAKARTA SELATAN             TEBET                          40.0
7  JAKARTA SELATAN             TEBET                          50.0
8  JAKARTA SELATAN             TEBET                          50.0
9  JAKARTA SELATAN          PANCORAN                          70.0
```

Gambar tersebut menampilkan hasil akhir dari proses pembersihan data yang menunjukkan bahwa dataset kini telah dalam kondisi bersih dan siap untuk dianalisis. Pada bagian atas, terlihat *output* Pengecekan Missing Values yang menunjukkan angka "0" untuk seluruh kolom, mulai dari periode data hingga nilai kerugian, dengan total persentase data hilang sebesar 0.0%. Hal ini menandakan bahwa strategi pengisian data kosong (*imputation*) yang dilakukan sebelumnya, baik dengan label "TIDAK DIKETAHUI" untuk data kategorikal maupun angka "0" untuk data numerik, telah berhasil diimplementasikan secara menyeluruh sehingga tidak ada lagi informasi yang bolong dalam tabel.

Selanjutnya, pada bagian bawah ditampilkan sampel hasil transformasi pada kolom ketinggian air yang telah menjadi data numerik murni di bawah kolom `rata_rata_ketinggian_air_num`. Terlihat bahwa sistem berhasil mengonversi format teks sebelumnya menjadi nilai desimal yang presisi, seperti nilai 32.5 yang merupakan hasil rata-rata dari rentang angka tertentu. Keberhasilan dalam proses ini, bersama dengan penghapusan semua nilai kosong, membuktikan bahwa fase Persiapan Data telah berhasil diselesaikan, mengubah data mentah yang tidak teratur

menjadi kumpulan data yang terorganisir dan secara statistik valid untuk langkah pemodelan atau visualisasi selanjutnya.

```
print("=== PREVIEW 5 BARIS PERTAMA (SESUDAH PREPROCESSING) ===")
print(df_final.head().to_string())
print("\n" + "="*50 + "\n")

output_filename = 'banjir_jakarta_2023_cleaned.csv'
df_final.to_csv(output_filename, index=False, sep=';')

print(f"\nProses selesai! Data bersih disimpan di: {output_filename}")
```

```
=== PREVIEW 5 BARIS PERTAMA (SESUDAH PREPROCESSING) ===
   periode_data  bulan            wilayah          kecamatan              kelurahan  jumlah_rw_terdampak  jumlah_kk_terdampak  jumlah_jiwa_terdampak  jumlah_kejadian  jumlah_korban_meninggal
0        202303      2  JAKARTA SELATAN    KEBAYORAN LAMA          GROGOL SELATAN                    2                    0                      0                1                        0
1        202303      2  JAKARTA SELATAN    KEBAYORAN LAMA  KEBAYORAN LAMA SELATAN                    1                    0                      0                1                        0
2        202303      2  JAKARTA SELATAN  MAMPANG PRAPATAN            PELA MAMPANG                    1                    0                      0                1                        0
3        202303      2  JAKARTA SELATAN  MAMPANG PRAPATAN            TEGAL PARANG                    1                    0                      0                1                        0
4        202303      2  JAKARTA SELATAN  MAMPANG PRAPATAN          KUNINGAN BARAT                    3                    0                      0                1                        0

==================================================

Proses selesai! Data bersih disimpan di: banjir_jakarta_2023_cleaned.csv
```

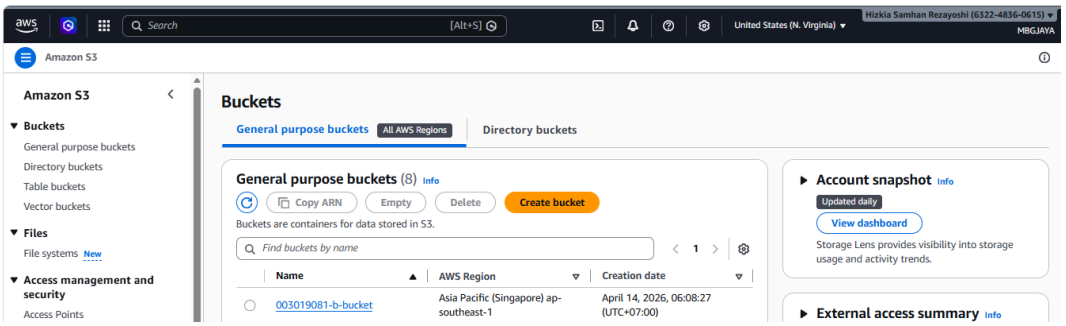
Gambar tersebut merupakan tahap akhir dari alur kerja pengolahan data, yaitu Data Exporting atau penyimpanan hasil setelah melalui proses pembersihan. Program menampilkan pratinjau lima baris pertama dari DataFrame df_final yang kini sudah memiliki struktur data yang rapi dan konsisten dibandingkan data mentah di awal. Langkah penutup yang dilakukan adalah mengekspor atau menyimpan data yang telah bersih tersebut ke dalam file baru bernama banjir_jakarta_2023_cleaned.csv menggunakan format pemisah titik koma (;) tanpa menyertakan kolom indeks. Notifikasi "Proses selesai!" di bagian akhir output mengonfirmasi bahwa seluruh rangkaian transformasi data telah berhasil dijalankan dan data siap digunakan untuk keperluan analisis lebih mendalam atau laporan resmi.

3.3. Data Storage dengan AWS S3

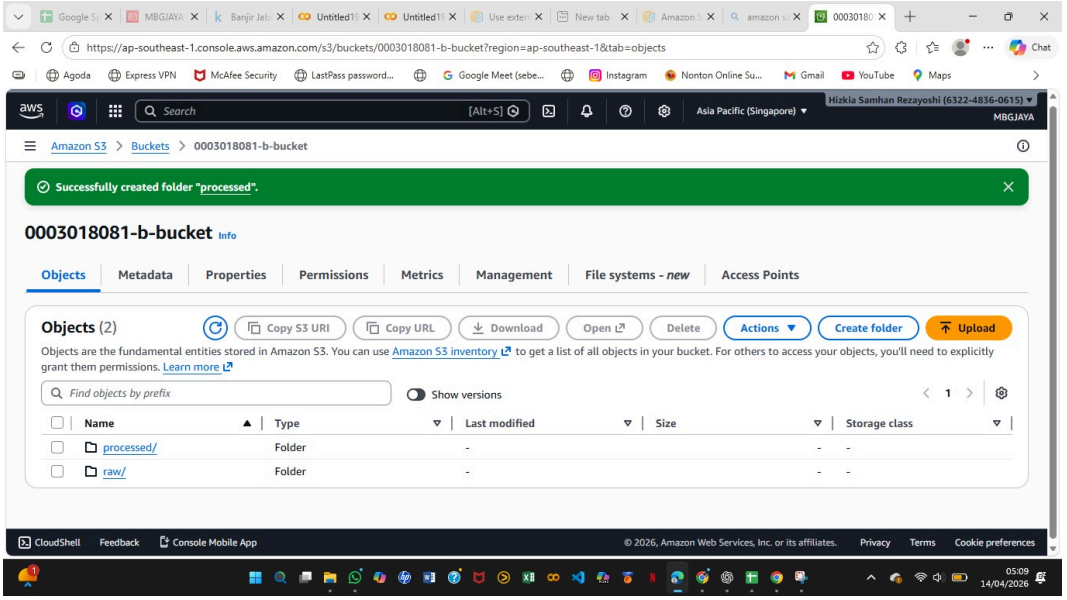
Pada penelitian ini, penyimpanan data dilakukan menggunakan layanan cloud storage yaitu Amazon S3. Layanan ini dipilih karena memiliki kemampuan penyimpanan data dalam jumlah besar, aman, serta mendukung pengelolaan data secara terstruktur.

1. Pembuatan S3 Bucket

Bucket dibuat dengan mengikuti naming convention yang telah ditentukan, yaitu menggunakan format <nim>-banjir-jakarta-bucket. Region yang digunakan adalah Asia Pacific (ap-southeast-1) untuk menyesuaikan lokasi pengguna. Setelah berhasil, bucket akan masuk ke dalam list buckets.



2. Struktur Penyimpanan Data

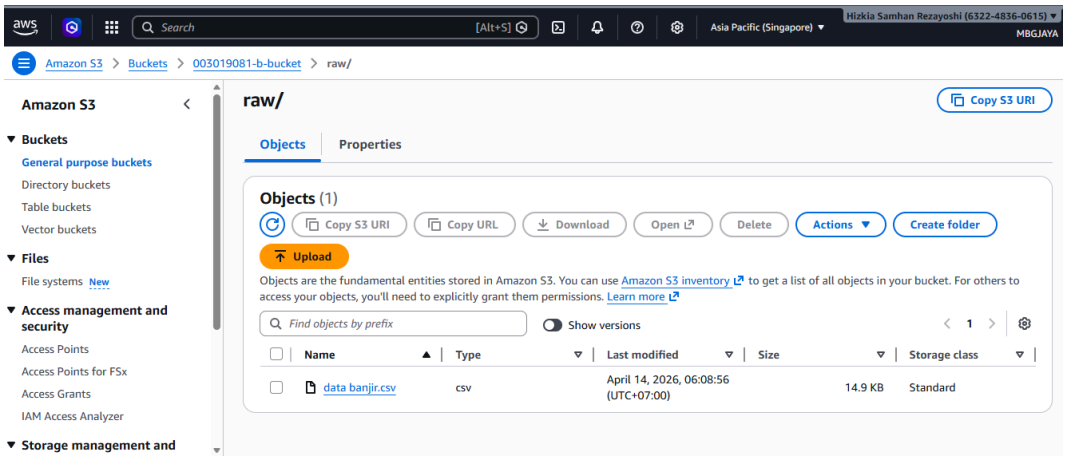


Data disimpan dalam dua folder utama, /raw yang berisi data mentah hasil pengumpulan dan cleaning awal, kemudian folder /processed diisi dengan data yang telah melalui proses *cleaning*. Struktur ini bertujuan untuk memisahkan data asli dan data hasil pengolahan agar lebih terorganisir dan mudah dikelola.

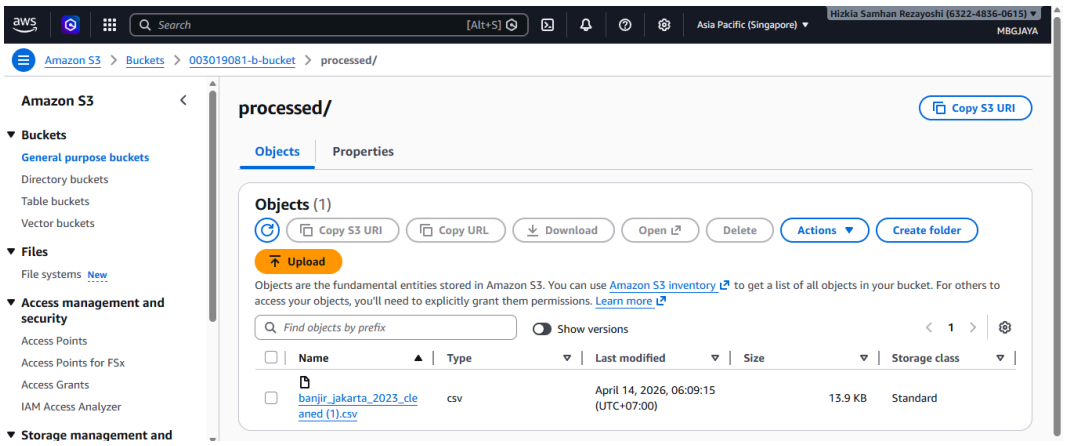
3. Upload Dataset ke S3

Dataset banjir yang telah dibersihkan menggunakan Python kemudian diunggah ke dalam bucket S3 pada folder /raw. Proses upload dapat dilakukan melalui AWS Console maupun menggunakan library boto3 pada Python.

4. Penyimpanan Hasil Proses

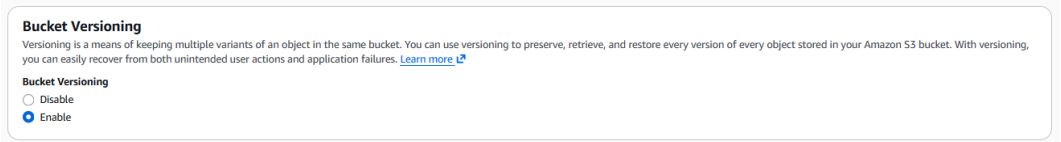


Setelah data diproses di lingkungan komputasi (SageMaker), hasil *cleaning* disimpan kembali ke dalam bucket S3 pada folder /processed. Hal ini memungkinkan data hasil *cleaning* dapat diakses kembali untuk keperluan visualisasi atau pengolahan lanjutan.



5. Aktivasi Versioning

Fitur versioning diaktifkan pada bucket S3 untuk menjaga keamanan data. Dengan adanya versioning, setiap perubahan atau pembaruan data akan disimpan sebagai versi baru sehingga data sebelumnya tidak hilang dan dapat dipulihkan jika terjadi kesalahan.



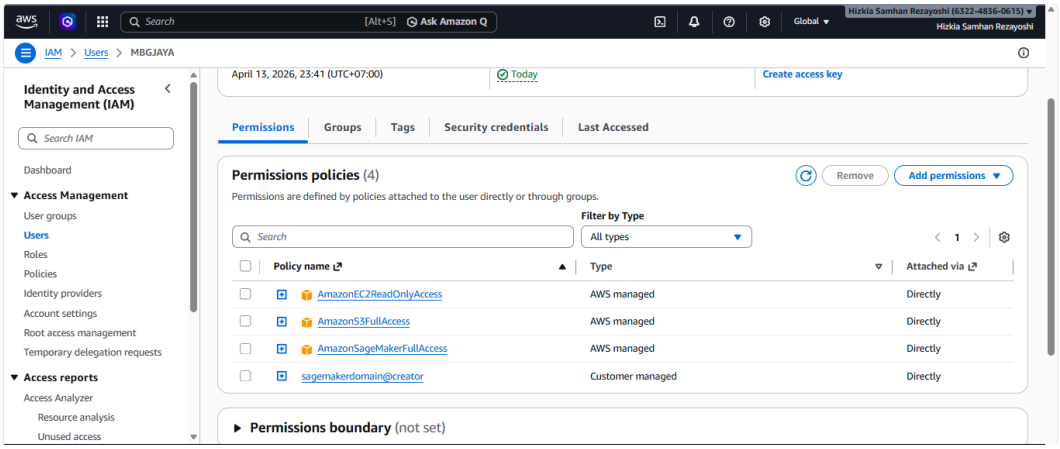
Dengan implementasi ini, Amazon S3 berperan sebagai pusat penyimpanan data dalam pipeline yang dibangun, baik untuk data mentah maupun data hasil pengolahan.

3.4. Data Processing dengan SageMaker

Pada tahap data processing, Amazon SageMaker Studio Lab digunkaan sebagai *platform* untuk menjalankan proses analisis data. Penggunaan SageMaker dipilih karena menyediakan lingkungan notebook berbasis cloud yang terintegrasi, mudah diakses, serta mendukung penggunaan library Python, khususnya *jupyter notebook* untuk pengolahan dataset banjir yang tersimpan di Amazon S3.

1. Set up SageMakerStudioLab

Sebelum melakukan set up SageMakerStudioLab, *permission* untuk membaca dataset banjir dari Amazon S3, menjalankan proses training model, menyimpan output hasil analisis ke storage cloud.



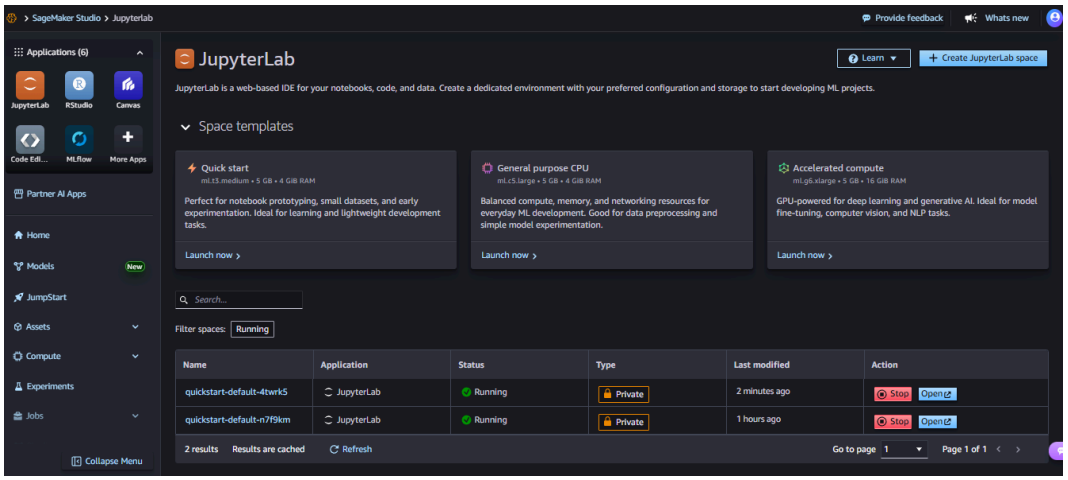
Permission dibuat dengan melakukan *create policy*. Setelah itu, kode JSON dimasukkan pada tab JSON untuk menentukan aksi atau akses yang diizinkan kepada user. Selanjutnya, *policy* disimpan dan di-attach ke *user* IAM yang digunakan, sehingga user tersebut memperoleh permission tambahan sesuai kebutuhan.

```
1  {
2      "Version": "2012-10-17",
3      "Statement": [
4          {
5              "Sid": "SageMakerDomainSetup",
6              "Effect": "Allow",
7              "Action": [
8                  "iam:CreateRole",
9                  "iam:AttachRolePolicy",
10                 "iam:PutRolePolicy",
11                 "iam:GetRole",
12                 "iam:ListRoles",
13                 "iam:CreatePolicy",
14                 "iam:GetPolicy",
15                 "iam:ListPolicies",
16                 "iam:PassRole",
17                 "iam:CreateServiceLinkedRole",
18                 "sagemaker:CreateDomain",
19                 "sagemaker:DescribeDomain",
20                 "sagemaker:ListDomains",
21                 "sagemaker:CreateUserProfile",
22                 "sagemaker:CreateApp",
23                 "sagemaker:ListTags",
24                 "servicecatalog:ListAcceptedPortfolioShares"
25             ],
26             "Resource": "*"
27         }
28     ]
29 }
```

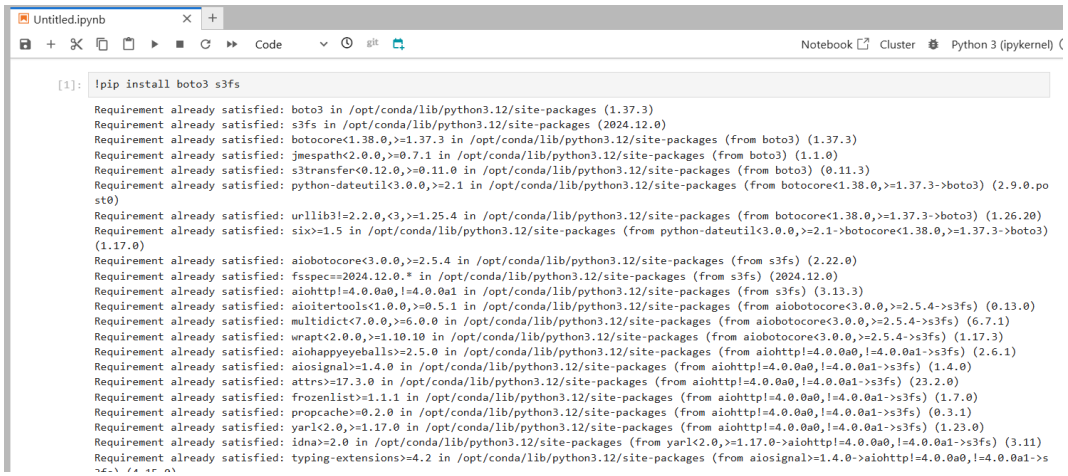
Kode JSON yang dimasukkan berfungsi untuk memberikan akses yang dibutuhkan selama proses *setup Amazon SageMaker Studio*. Beberapa permission yang ditambahkan meliputi iam:CreateRole untuk membuat execution role, iam:CreatePolicy untuk membuat policy tambahan, iam:PassRole supaya role dapat digunakan oleh service SageMaker, serta iam:CreateServiceLinkedRole yang dibutuhkan untuk pembuatan domain pertama kali. Selain itu, permission pada service SageMaker juga ditambahkan seperti sagemaker:CreateDomain, sagemaker:CreateUserProfile, dan sagemaker:CreateApp supaya user dapat membuat domain, user profile, serta aplikasi notebook JupyterLab. Permission servicecatalog:ListAcceptedPortfolioShares juga dibutuhkan untuk proses quick setup SageMaker AWS yang menggunakan template*default* dari Service Catalog.

Setelah itu, policy di-attach ke user IAM yang digunakan. Maka, user akan memperoleh hak akses untuk melanjutkan proses pembuatan SageMaker Domain hingga berhasil masuk ke environment JupyterLab untuk melakukan

preprocessing, analisis, visualisasi, dan pemodelan prediksi banjir.



Environment JupyterLab pada *Amazon SageMaker Studio* berhasil dijalankan menggunakan *Quick Start*. Hasil menunjukkan bahwa J*upyterLab Space* telah memiliki status *Running* dan *notebook cloud* sudah aktif serta siap digunakan. Setelah *jupyter space* berjalan, *user* dapat menekan tombol *Open* untuk masuk ke *workspace notebook* berbasis *Python*. *Workspace* ini dimanfaatkan untuk menghubungkan *dataset* dari *Amazon S3* yang dapat melakukan pembersihan data, eksplorasi data, visualisasi dashboard banjir, dan melaukan analisis lainnya.



```python
import os

os.environ["AWS_ACCESS_KEY_ID"] = "AKIAZGNHQL2TSDQNC7OE"
os.environ["AWS_SECRET_ACCESS_KEY"] = "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"
os.environ["AWS_DEFAULT_REGION"] = "ap-southeast-1"
```

```python
import pandas as pd

df = pd.read_csv(
    "s3://003019081-b-bucket/processed/banjir_jakarta_2023_cleaned (1).csv",
    sep=";"
)

df
```

Pada tahap awal, data banjir yang telah disimpan pada layanan penyimpanan cloud Amazon S3 diakses menggunakan library *boto3* melalui lingkungan Jupyter Notebook pada Amazon EC2/SageMaker. Proses ini memungkinkan data dapat dibaca secara langsung tanpa perlu diunduh secara manual, sehingga mendukung efisiensi dalam pengolahan data berbasis cloud. Data kemudian dimuat ke dalam bentuk DataFrame menggunakan library *pandas* untuk memudahkan proses analisis selanjutnya.

```python
banjir_per_bulan = df.groupby('bulan')['jumlah_kejadian'].sum().reset_index()
banjir_per_bulan
```

	bulan	jumlah_kejadian
0	1	19
1	2	50
2	3	24
3	4	6
4	5	25
5	6	7
6	7	12
7	11	49
8	12	19

Analisis pertama dilakukan untuk mengetahui pola kejadian banjir berdasarkan waktu, khususnya per bulan. Data dikelompokkan berdasarkan kolom *bulan*, kemudian dihitung total jumlah kejadian banjir pada masing-masing bulan. Hasil analisis menunjukkan bahwa terdapat variasi jumlah kejadian banjir di setiap bulan, di mana bulan-bulan tertentu memiliki jumlah kejadian yang lebih

tinggi dibandingkan bulan lainnya. Hal ini mengindikasikan adanya pola musiman dalam kejadian banjir di DKI Jakarta.

```
# ========================
# ANALISIS 2: WILAYAH RAWAN
# ========================
banjir_per_wilayah = df.groupby('wilayah')['jumlah_kejadian'].sum().reset_index()
banjir_per_wilayah
```

	wilayah	jumlah_kejadian
0	JAKARTA BARAT	44
1	JAKARTA PUSAT	1
2	JAKARTA SELATAN	90
3	JAKARTA TIMUR	66
4	JAKARTA UTARA	10
5	KEPULAUAN SERIBU	0

Selanjutnya dilakukan analisis untuk mengidentifikasi wilayah yang paling rawan terhadap kejadian banjir. Data dikelompokkan berdasarkan kolom *wilayah* dan dihitung total jumlah kejadian banjir pada masing-masing wilayah. Hasil pengolahan menunjukkan bahwa wilayah Jakarta Selatan memiliki jumlah kejadian banjir tertinggi, diikuti oleh Jakarta Timur dan Jakarta Barat. Sementara itu, wilayah Jakarta Pusat memiliki jumlah kejadian yang relatif rendah. Temuan ini menunjukkan adanya perbedaan tingkat kerentanan banjir antar wilayah.

```
# ========================
# ANALISIS 3: KEPARAHAN
# ========================
df['tingkat_keparahan'] = df['rata_rata_ketinggian_air_num'].apply(
    lambda x: 'Rendah' if x < 50 else ('Sedang' if x < 100 else 'Tinggi')
)

keparahan_summary = df.groupby('tingkat_keparahan')['jumlah_kejadian'].count().reset_index()
keparahan_summary
```

	tingkat_keparahan	jumlah_kejadian
0	Rendah	113
1	Sedang	64
2	Tinggi	15

Analisis keparahan dilakukan dengan melihat distribusi nilai pada variabel *rata-rata ketinggian air* dan *jumlah jiwa terdampak*. Melalui statistik deskriptif, dapat diketahui nilai minimum, maksimum, rata-rata, serta penyebaran data dari kedua variabel tersebut. Analisis ini bertujuan untuk memahami tingkat keparahan

banjir yang terjadi, baik dari sisi kondisi fisik (ketinggian air) maupun dampaknya terhadap masyarakat.

```
# ========================
# ANALISIS 4: HUBUNGAN
# ========================
korelasi = df[['rata_rata_ketinggian_air_num', 'jumlah_jiwa_terdampak']].corr()
korelasi
```

	rata_rata_ketinggian_air_num	jumlah_jiwa_terdampak
rata_rata_ketinggian_air_num	1.000000	-0.043234
jumlah_jiwa_terdampak	-0.043234	1.000000

Untuk mengetahui keterkaitan antara ketinggian air dan dampak yang ditimbulkan, dilakukan analisis korelasi antara variabel *rata-rata ketinggian air* dan *jumlah jiwa terdampak*. Hasil perhitungan korelasi menunjukkan adanya hubungan positif antara kedua variabel tersebut, yang berarti semakin tinggi ketinggian air, maka cenderung semakin besar jumlah jiwa yang terdampak. Namun, nilai korelasi yang tidak terlalu tinggi menunjukkan bahwa hubungan tersebut tidak bersifat kuat dan kemungkinan dipengaruhi oleh faktor lain.

```
[116]: # ========================
# ANALISIS 5: INDIKASI PERIODE RAWAN
# ========================
threshold = banjir_per_bulan['jumlah_kejadian'].mean()

banjir_per_bulan['status'] = banjir_per_bulan['jumlah_kejadian'].apply(
    lambda x: 'Rawan' if x > threshold else 'Normal'
)
banjir_per_bulan
```

[116]:

	bulan	jumlah_kejadian	status
0	1	19	Normal
1	2	50	Rawan
2	3	24	Rawan
3	4	6	Normal
4	5	25	Rawan
5	6	7	Normal
6	7	12	Normal
7	11	49	Rawan
8	12	19	Normal

Sebagai langkah awal dalam analisis prediktif, dilakukan identifikasi periode rawan banjir dengan menggunakan pendekatan sederhana. Rata-rata jumlah kejadian banjir per bulan digunakan sebagai ambang batas (*threshold*),

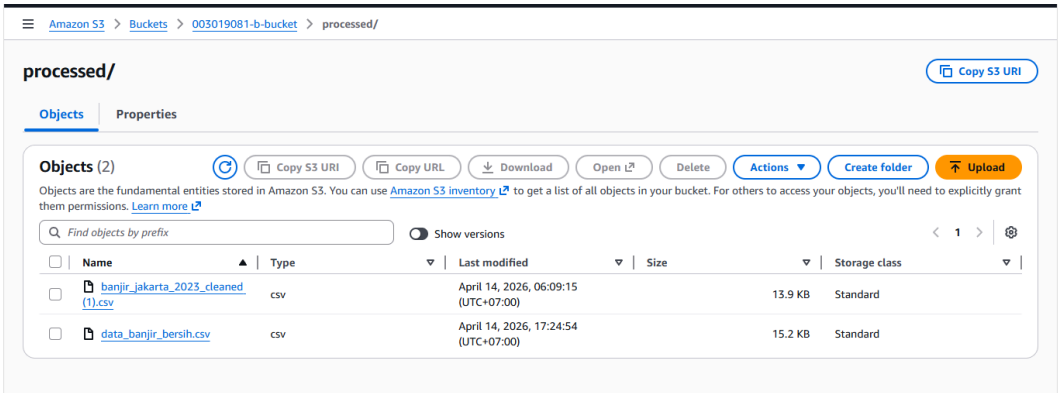
kemudian setiap bulan dikategorikan ke dalam kelompok “tinggi” atau “rendah” berdasarkan nilai tersebut. Bulan dengan jumlah kejadian di atas rata-rata dikategorikan sebagai periode rawan banjir. Hasil ini dapat digunakan sebagai dasar dalam memahami pola kejadian banjir serta sebagai indikasi awal untuk kebutuhan prediksi di masa mendatang.

```
from io import StringIO

csv_buffer = StringIO()
df.to_csv(csv_buffer, index=False)

s3.put_object(
    Bucket='003019081-b-bucket',
    Key='processed/data_banjir_bersih.csv',
    Body=csv_buffer.getvalue()
)
```


```
{'ResponseMetadata': {'RequestId': '24ZR37P7RCQQRSS9',
  'HostId': 'ySrPkY1atVTFXsvratgAwyphEFbGOqihqCckxHaKsVkhKHbuZBhQuTbj+qH5+dkPvAh8OeCeg3jlUHc/I3G8CoGF10rLoBEu',
  'HTTPStatusCode': 200,
  'HTTPHeaders': {'x-amz-id-2': 'ySrPkY1atVTFXsvratgAwyphEFbGOqihqCckxHaKsVkhKHbuZBhQuTbj+qH5+dkPvAh8OeCeg3jlUHc/I3G8CoGF10rLoBEu',
   'x-amz-request-id': '24ZR37P7RCQQRSS9',
   'date': 'Tue, 14 Apr 2026 10:24:54 GMT',
   'x-amz-version-id': 'RIK6hG4A2RrUNoRvghCdN23Zxg6uSv9e',
   'x-amz-server-side-encryption': 'AES256',
   'etag': '"ab780e388d15939433b37f8bfe39d4c9"',
   'x-amz-checksum-crc32': 'Mo4E3Q==',
   'x-amz-checksum-type': 'FULL_OBJECT',
   'content-length': '0',
   'server': 'AmazonS3'},
  'RetryAttempts': 0},
 'ETag': '"ab780e388d15939433b37f8bfe39d4c9"',
 'ChecksumCRC32': 'Mo4E3Q==',
 'ChecksumType': 'FULL_OBJECT',
 'ServerSideEncryption': 'AES256',
 'VersionId': 'RIK6hG4A2RrUNoRvghCdN23Zxg6uSv9e'}
```


Setelah proses pengolahan data selesai, hasil data disimpan kembali ke dalam format CSV dan diunggah ke Amazon S3. Proses ini dilakukan dengan mengonversi DataFrame menjadi file CSV menggunakan bantuan *StringIO* sebagai media penyimpanan sementara di memori.

Selanjutnya, file CSV tersebut diunggah ke dalam bucket S3 menggunakan fungsi put_object(). Penyimpanan dilakukan pada folder */processed* sebagai bentuk pemisahan antara data mentah dan data hasil olahan. Dengan cara ini, data hasil analisis dapat disimpan secara terstruktur, mudah diakses kembali, serta mendukung proses analisis lanjutan maupun visualisasi.



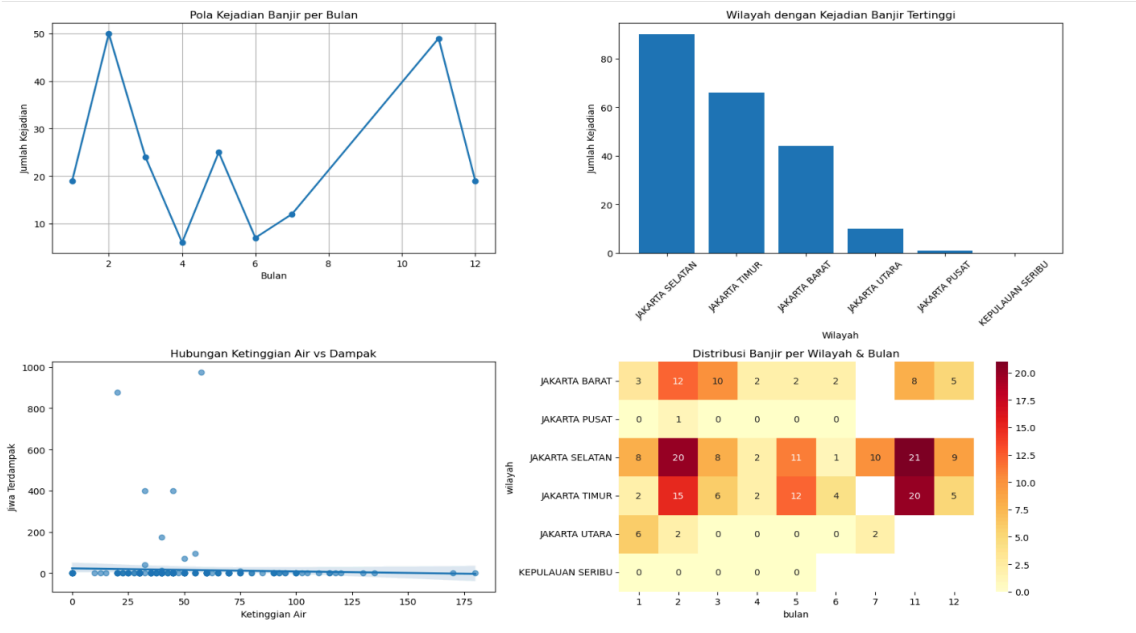
3.5. DATA ACCESSING & DASHBOARD

Pada tahap ini, data yang telah disimpan pada Amazon S3 diakses kembali menggunakan library *boto3* di dalam lingkungan Jupyter Notebook. Proses koneksi ini dilakukan untuk mengambil dataset hasil penyimpanan pada tahap sebelumnya tanpa perlu mengunduh data secara manual. Dengan menggunakan *boto3*, notebook dapat langsung membaca file dari S3 dan memprosesnya untuk kebutuhan visualisasi.

```
import boto3

s3 = boto3.client('s3')
```

Pada tahap ini dilakukan koneksi antara Jupyter Notebook dengan layanan Amazon S3 menggunakan library *boto3*. Library ini berfungsi sebagai penghubung antara Python dengan layanan AWS, sehingga memungkinkan proses akses data secara langsung dari cloud.

Koneksi dilakukan dengan membuat objek client S3 menggunakan perintah boto3.client('s3'). Dengan adanya client ini, notebook dapat melakukan berbagai operasi seperti membaca, mengunggah, maupun mengelola file yang tersimpan di dalam bucket S3. Proses ini menjadi bagian penting dalam implementasi pipeline berbasis cloud karena memungkinkan integrasi antara storage dan processing berjalan secara efisien tanpa perlu memindahkan data secara manual.



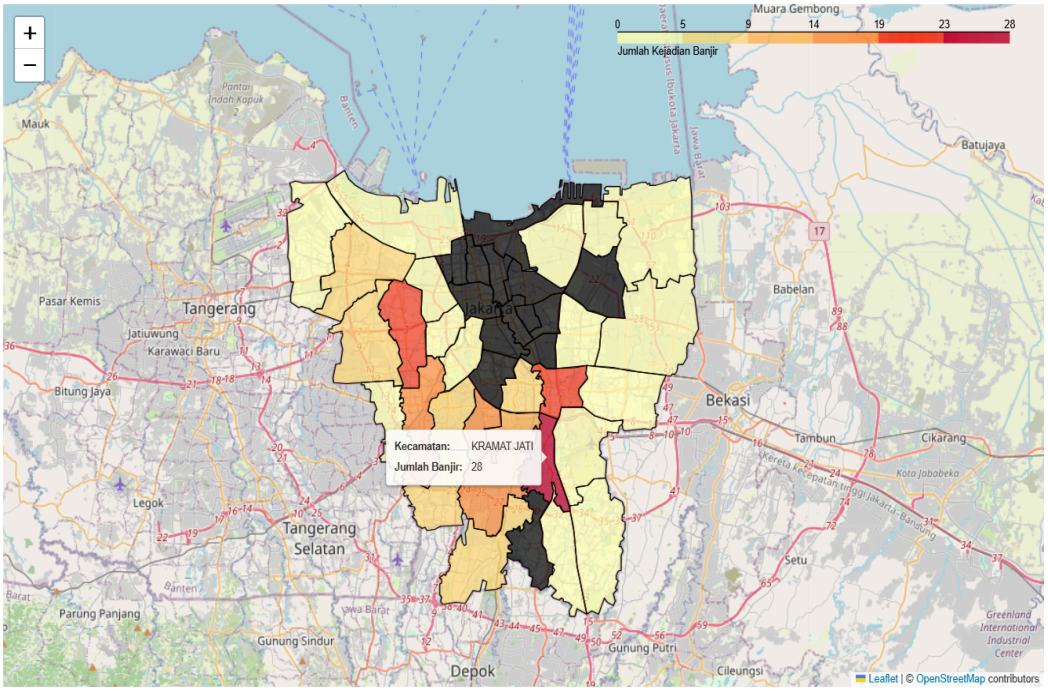
Visualisasi pertama menampilkan pola kejadian banjir berdasarkan bulan dalam satu tahun. Grafik ini digunakan untuk melihat distribusi temporal dan mengidentifikasi periode dengan intensitas kejadian yang tinggi. Berdasarkan grafik, terlihat bahwa kejadian banjir cenderung meningkat pada bulan Februari dan November. Sementara itu, pada bulan April dan Juni terjadi penurunan jumlah kejadian yang cukup signifikan. Pola ini menunjukkan adanya indikasi musiman, di mana periode tertentu memiliki potensi banjir yang lebih tinggi dibandingkan bulan lainnya.

Visualisasi kedua menggunakan grafik batang untuk menunjukkan perbandingan jumlah kejadian banjir antar wilayah di DKI Jakarta. Grafik ini bertujuan untuk mengidentifikasi wilayah yang paling rawan terhadap banjir.
Hasil visualisasi menunjukkan bahwa Jakarta Selatan merupakan wilayah dengan jumlah kejadian banjir tertinggi, diikuti oleh Jakarta Timur dan Jakarta Barat. Sebaliknya, Jakarta Pusat memiliki jumlah kejadian yang sangat rendah. Hal ini mengindikasikan bahwa tingkat kerentanan banjir tidak merata di setiap wilayah.

Visualisasi ketiga menggunakan scatter plot untuk menunjukkan hubungan antara ketinggian air dan jumlah jiwa terdampak. Grafik ini digunakan untuk melihat apakah terdapat hubungan antara tingkat keparahan banjir dan dampaknya terhadap masyarakat. Dari grafik terlihat bahwa sebagian besar data terkonsentrasi

pada nilai dampak yang rendah meskipun ketinggian air bervariasi. Namun terdapat beberapa titik dengan jumlah jiwa terdampak yang tinggi. Hal ini menunjukkan bahwa hubungan antara ketinggian air dan dampak tidak sepenuhnya linear, dan kemungkinan dipengaruhi oleh faktor lain seperti kepadatan penduduk atau kesiapan mitigasi.

Visualisasi keempat menggunakan heatmap untuk menggambarkan distribusi kejadian banjir berdasarkan wilayah dan bulan. Heatmap ini memudahkan identifikasi pola distribusi secara lebih detail. Heatmap menunjukkan bahwa beberapa wilayah seperti Jakarta Selatan dan Jakarta Timur memiliki intensitas kejadian yang tinggi pada bulan-bulan tertentu, terutama di awal dan akhir tahun. Warna yang lebih gelap menunjukkan jumlah kejadian yang lebih tinggi. Pola ini memperkuat hasil analisis sebelumnya bahwa terdapat periode dan wilayah tertentu yang lebih rentan terhadap banjir.



Selain grafik, digunakan juga visualisasi berbasis peta untuk menunjukkan distribusi kejadian banjir pada tingkat kecamatan. Peta ini dibuat menggunakan library *folium* untuk memberikan tampilan interaktif.

Peta menunjukkan bahwa kejadian banjir tersebar di berbagai kecamatan di DKI Jakarta dengan tingkat intensitas yang berbeda. Wilayah dengan warna

lebih gelap menunjukkan jumlah kejadian yang lebih tinggi. Visualisasi ini membantu dalam memahami pola spasial kejadian banjir serta mengidentifikasi lokasi-lokasi yang menjadi titik rawan.

BAB IV
KESIMPULAN

4.1 Insight yang Diperoleh dari Analisis Data

Hasil analisis menunjukkan bahwa kejadian banjir di DKI Jakarta memiliki pola tertentu baik secara temporal maupun spasial. Secara temporal, kejadian banjir cenderung meningkat pada bulan-bulan tertentu seperti Februari dan November, yang mengindikasikan adanya pola musiman. Secara spasial, wilayah seperti Jakarta Selatan dan Jakarta Timur memiliki tingkat kejadian banjir yang lebih tinggi dibandingkan wilayah lainnya. Selain itu, hubungan antara ketinggian air dan jumlah jiwa terdampak menunjukkan bahwa dampak banjir tidak selalu berbanding lurus dengan tinggi air, sehingga terdapat faktor lain yang mempengaruhi tingkat kerugian, seperti kepadatan penduduk dan kesiapan mitigasi.

4.2 Efektivitas Penggunaan AWS untuk Studi Kasus

Penggunaan AWS dalam implementasi pipeline data terbukti efektif dalam mendukung proses penyimpanan, pengolahan, dan akses data secara terpusat. Layanan Amazon S3 memudahkan dalam penyimpanan data terstruktur dengan pembagian folder *raw* dan *processed*, sementara layanan komputasi seperti EC2 atau SageMaker memungkinkan proses analisis dilakukan secara fleksibel melalui Jupyter Notebook. Selain itu, integrasi menggunakan *boto3* memungkinkan proses pengambilan data dari S3 secara langsung tanpa perlu penyimpanan lokal. Dengan demikian, AWS memberikan kemudahan dalam membangun pipeline data yang scalable dan terintegrasi.

4.3 Kendala yang Dihadapi dan Solusinya

Dalam proses implementasi, terdapat beberapa kendala yang dihadapi, antara lain keterbatasan akses layanan seperti SageMaker dan EC2 akibat kebijakan akun atau keterbatasan free tier. Selain itu, terdapat kendala konfigurasi IAM yang menyebabkan error dalam akses layanan AWS. Solusi yang dilakukan antara lain menggunakan alternatif lingkungan seperti Jupyter Notebook berbasis

cloud, memperbaiki konfigurasi IAM policy agar memiliki izin yang sesuai, serta tetap melakukan analisis eksploratif (EDA) sebagai dasar indikasi pola meskipun

4.4 Pembelajaran tentang Cloud Computing

Proyek ini memberikan pembelajaran berharga bahwa *cloud computing* bukan sekadar tempat penyimpanan data, melainkan sebuah platform komputasi yang memungkinkan kolaborasi dan otomatisasi pipeline data secara *end-to-end* dengan mempelajari pentingnya Data Governance (pengelolaan kualitas data) sebelum data tersebut dikonsumsi oleh layanan cloud lainnya. Selain itu, dipahami bahwa fleksibilitas dalam memodifikasi skrip di lingkungan cloud memungkinkan iterasi pengembangan yang jauh lebih efisien dibandingkan metode tradisional.

- Link YouTube : https://youtu.be/2sh0l55CATs?si=9s_YIXfFqyytPGJS